Diseño y Análisis de Algoritmos

Ayudantía 5: Programación Dinámica

Ayud. Ferran Rojas Andreu

Enunciado

Se dispone de monedas con denominaciones $D = \{1, 3, 4\}$ (hay infinitas monedas de cada valor) y se quiere pagar exactamente un monto $A \geq 0$. El objetivo es usar la *mínima cantidad de monedas*.

(a) Para el ejemplo $A = 6$, calcule a mano el valor óptimo $F(i)$ (mínimo número de monedas para pagar el monto i) para $i = 0, 1, \ldots, 6$, e indique el valor final $F(6)$.

(b) Defina la recurrencia $F(i)$ para todo $i \geq 0$, indicando el caso base.

(c) Usando pseudocódigo, implemente MinCoins($D[1..k]$, A) que retorne $F(A)$ en tiempo $O(A \cdot k)$ y espacio adicional $O(A)$. Justifique brevemente las cotas de complejidad.

Supuestos: (i) la denominación 1 siempre está disponible, por lo que todo monto es pagable; (ii) el orden de las monedas no importa, solo la cantidad usada.

Enunciado 2: Subsecuencia creciente más larga (LIS)

Dado un arreglo $A[1..n]$ de números enteros, una *subsecuencia* de A es cualquier secuencia obtenida eliminando cero o más elementos de A sin alterar el orden relativo de los restantes. Se desea encontrar la *subsecuencia creciente más larga* (*Longest Increasing Subsequence*, LIS), es decir, el máximo k tal que existen índices

$$i_1 < i_2 < \cdots < i_k$$

con

$$A[i_1] < A[i_2] < \cdots < A[i_k].$$

Considere el arreglo

$$A = \langle 10,\ 9,\ 2,\ 5,\ 3,\ 7,\ 101,\ 18 \rangle.$$

(a) Defina $F(i)$ como la *longitud de la subsecuencia creciente más larga que termina en* $A[i]$. Calcule a mano $F(i)$ para $i = 1, \ldots, 8$, indicando el caso base y el valor final del LIS.

(b) Escriba la recurrencia $F(i)$ para todo $1 \leq i \leq n$, indicando el caso base.

(c) Usando pseudocódigo, implemente LIS($A[1..n]$, n) que retorne la longitud del LIS en tiempo $O(n^2)$ y espacio adicional $O(n)$. Justifique brevemente las cotas de complejidad.

Nota: puede incluir (opcional) un arreglo de predecesores para reconstruir una subsecuencia concreta.

Enunciado 3: Knapsack 0/1

Se tienen n objetos, donde cada objeto i tiene un peso $w_i > 0$ y un valor $v_i > 0$, y una mochila de capacidad $W \geq 0$. En la variante *0/1*, cada objeto se toma *una vez* o *no se toma* (no se puede fraccionar). Se desea seleccionar un subconjunto $S \subseteq \{1, \ldots, n\}$ que maximice el valor total

$$\sum_{i \in S} v_i \qquad \text{sujeto a} \qquad \sum_{i \in S} w_i \leq W.$$

Considere los datos

i	1	2	3	4
w_i	2	3	4	5
v_i	3	4	5	6

$W = 8.$

(a) Defina $F(i, j)$ como el *máximo valor alcanzable usando solo los primeros i objetos con capacidad total j*. Calcule a mano la tabla $F(i, j)$ para $i = 1, \ldots, 4$ y $j = 0, \ldots, 8$, indicando los casos base y el valor final $F(4, 8)$.

(b) Escriba la recurrencia de $F(i, j)$ indicando los dos casos: (1) el objeto i no cabe en la capacidad j, y (2) cabe, comparando *no tomarlo* vs. *tomarlo*. Indique los casos base.

(c) Usando pseudocódigo, implemente KNAPSACK01($w[1..n]$, $v[1..n]$, n, W) que retorne el valor óptimo en tiempo $O(n\,W)$ y espacio $O(n\,W)$. Justifique brevemente las cotas de complejidad.

Nota: puede incluir (opcional) una matriz de decisión para reconstruir el subconjunto óptimo.